
Lecture 4: Data Analysis and Preprocessing

Part 2

Md. Shahriar Hussain
ECE Department, NSU



Data Frames: Sorting

- We can sort the data by a value in the column. By default the sorting will occur in ascending order and a new data frame is return.

```
In [ ]: # Create a new data frame from the original sorted by the column Salary  
df_sorted = df.sort_values( by ='service')  
df_sorted.head()
```

```
Out[ ]:
```

	rank	discipline	phd	service	sex	salary
55	AsstProf	A	2	0	Female	72500
23	AsstProf	A	2	0	Male	85000
43	AsstProf	B	5	0	Female	77000
17	AsstProf	B	4	0	Male	92000
12	AsstProf	B	1	0	Male	88000



Data Frames: Sorting

- We can sort the data using 2 or more columns:

```
In [ ]: df_sorted = df.sort_values( by=['service', 'salary'], ascending = [True, False])  
df_sorted.head(10)
```

```
Out[ ]:
```

	rank	discipline	phd	service	sex	salary
52	Prof	A	12	0	Female	105000
17	AsstProf	B	4	0	Male	92000
12	AsstProf	B	1	0	Male	88000
23	AsstProf	A	2	0	Male	85000
43	AsstProf	B	5	0	Female	77000
55	AsstProf	A	2	0	Female	72500
57	AsstProf	A	3	1	Female	72500
28	AsstProf	B	7	2	Male	91300
42	AsstProf	B	4	2	Female	80225
68	AsstProf	A	4	2	Female	77500



Handle missing/ null values: Data Cleaning

Deleting the row with missing data: Used for large dataset

- The rows which are having one or more columns values as null can also be dropped.
- Loss of a lot of information.
- Works poorly if the percentage of missing values is excessive in comparison to the complete dataset
- If the proportion of missing observations in data is small relative to the total number of observations, we can simply remove those observations.
- Deleting the rows containing missing values may lead to parting away with useful information or patterns

Deleting the column with missing data: Used for many features.

- If columns have more than half of the rows as null then the entire column can be dropped. The rows which are having one or more columns values as null can also be dropped



Handle missing/ null values

- Dataset with missing values

In [33]:

```
import pandas as pd
df = pd.read_csv("weather_data.csv", parse_dates=["day"])
df.set_index('day', inplace=True)
df
```

Out[33]:

	temperature	windspeed	event
day			
2017-01-01	32.0	6.0	Rain
2017-01-04	NaN	9.0	Sunny
2017-01-05	28.0	NaN	Snow
2017-01-06	NaN	7.0	NaN
2017-01-07	32.0	NaN	Rain
2017-01-08	NaN	NaN	Sunny
2017-01-09	NaN	NaN	NaN
2017-01-10	34.0	8.0	Cloudy
2017-01-11	40.0	12.0	Sunny



dropna() Method

- `dropna()` Method removes all rows with NULL values from the DataFrame

```
import pandas as pd
df = pd.read_csv('data.csv')
newdf = df.dropna()
```

- Syntax

`dataframe.dropna(axis, how, thresh, subset, inplace)`

axis: default 0.

0 and 'index' removes ROWS that contains NULL values

1 and 'columns' removes COLUMNS that contains NULL values



dropna() Method

- `how`: default 'any'.

Specifies whether to remove the row or column when ALL values are NULL, or if ANY value is NULL.

- `thresh`: Specifies the number of NOT NULL values required to keep the row.
- `subset`: specifies where to look for NULL values
- `inplace`: default False.

If True: the removing is done on the current DataFrame. If False: returns a copy where the removing is done.



dropna() Method

```
In [34]: new_df = df.dropna()  
new_df
```

```
Out[34]:
```

	temperature	windspeed	event
day			
2017-01-01	32.0	6.0	Rain
2017-01-10	34.0	8.0	Cloudy
2017-01-11	40.0	12.0	Sunny

```
In [45]: new_df = df.dropna(how='all')  
new_df
```

```
Out[45]:
```

	temperature	windspeed	event
day			
2017-01-01	32.0	6.0	Rain
2017-01-04	NaN	7.0	Sunny
2017-01-05	28.0	NaN	Snow
2017-01-06	NaN	7.0	NaN
2017-01-07	32.0	NaN	Rain
2017-01-08	NaN	NaN	Sunny
2017-01-10	34.0	8.0	Cloudy
2017-01-11	40.0	12.0	Sunny



dropna() Method

```
In [38]: new_df = df.dropna(thresh=2)
new_df
```

```
Out[38]:
```

	temperature	windspeed	event
day			
2017-01-01	32.0	6.0	Rain
2017-01-04	NaN	9.0	Sunny
2017-01-05	28.0	NaN	Snow
2017-01-07	32.0	NaN	Rain
2017-01-10	34.0	8.0	Cloudy
2017-01-11	40.0	12.0	Sunny



fillna() Method

- Filling missing values using fillna() method
- Fill with 0, mean median or any other value
- The fillna() method returns a new DataFrame object unless the inplace parameter is set to True, in that case the fillna() method does the replacing in the original DataFrame instead

```
In [34]: new_df = df.fillna(0)
         new_df
```

```
Out[34]:
```

	temperature	windspeed	event
day			
2017-01-01	32.0	6.0	Rain
2017-01-04	0.0	9.0	Sunny
2017-01-05	28.0	0.0	Snow
2017-01-06	0.0	7.0	0
2017-01-07	32.0	0.0	Rain
2017-01-08	0.0	0.0	Sunny
2017-01-09	0.0	0.0	0
2017-01-10	34.0	8.0	Cloudy
2017-01-11	40.0	12.0	Sunny



fillna() Method

- Filling missing values using `fillna()` method: using dictionary

In [35]:

```
new_df = df.fillna({  
    'temperature': 0,  
    'windspeed': 0,  
    'event': 'no event'  
})  
new_df
```

Out[35]:

	temperature	windspeed	event
day			
2017-01-01	32.0	6.0	Rain
2017-01-04	0.0	9.0	Sunny
2017-01-05	28.0	0.0	Snow
2017-01-06	0.0	7.0	no event
2017-01-07	32.0	0.0	Rain
2017-01-08	0.0	0.0	Sunny
2017-01-09	0.0	0.0	no event
2017-01-10	34.0	8.0	Cloudy



fillna() Method

- Replace NULL values with the value from the previous row

```
In [36]: new_df = df.fillna(method="ffill")  
new_df
```

```
Out[36]:
```

	temperature	windspeed	event
day			
2017-01-01	32.0	6.0	Rain
2017-01-04	32.0	9.0	Sunny
2017-01-05	28.0	9.0	Snow
2017-01-06	28.0	7.0	Snow
2017-01-07	32.0	7.0	Rain
2017-01-08	32.0	7.0	Sunny
2017-01-09	32.0	NaN	Sunny
2017-01-10	34.0	8.0	Cloudy
2017-01-11	40.0	12.0	Sunny



fillna() Method

- Replace NULL values with the value from the next row

```
In [62]: new_df = df.fillna(method="bfill")  
new_df
```

```
Out[62]:
```

	temperature	windspeed	event
day			
2017-01-01	32.0	6.0	Rain
2017-01-04	28.0	7.0	Sunny
2017-01-05	28.0	7.0	Snow
2017-01-06	32.0	7.0	Rain
2017-01-07	32.0	8.0	Rain
2017-01-08	34.0	8.0	Sunny
2017-01-09	34.0	8.0	Cloudy
2017-01-10	34.0	8.0	Cloudy
2017-01-11	40.0	12.0	Sunny



interpolation() method

- **Interpolation** is a method of filling **missing values** by estimating them based on the values of other data points.
- By default it uses Linear Interpolation method

In [43]:

```
new_df = df.interpolate()  
new_df
```

Out[43]:

	temperature	windspeed	event
day			
2017-01-01	32.000000	6.00	Rain
2017-01-04	30.000000	7.00	Sunny
2017-01-05	28.000000	7.00	Snow
2017-01-06	30.000000	7.00	NaN
2017-01-07	32.000000	7.25	Rain
2017-01-08	32.666667	7.50	Sunny
2017-01-09	33.333333	7.75	NaN
2017-01-10	34.000000	8.00	Cloudy
2017-01-11	40.000000	12.00	Sunny



interpolation() method

- Filling values on the basis of date

```
In [37]: new_df = df.interpolate(method="time")
new_df
```

```
Out[37]:
```

	temperature	windspeed	event
day			
2017-01-01	32.000000	6.00	Rain
2017-01-04	29.000000	9.00	Sunny
2017-01-05	28.000000	8.00	Snow
2017-01-06	30.000000	7.00	NaN
2017-01-07	32.000000	7.25	Rain
2017-01-08	32.666667	7.50	Sunny
2017-01-09	33.333333	7.75	NaN
2017-01-10	34.000000	8.00	Cloudy
2017-01-11	40.000000	12.00	Sunny



Reshaping an Array: Numpy *reshape()* method

- ***arrange(n)*** method creates an array with value 0 to n-1
- ***reshape()*** method converting the following 1-D array with 12 elements into a 2-D array.

```
In [27]: import numpy as np
```

```
In [28]: a = np.arange(12).reshape(3,4)
a
```

```
Out[28]: array([[ 0,  1,  2,  3],
                [ 4,  5,  6,  7],
                [ 8,  9, 10, 11]])
```



Flattening an array

- we can flatten an array either using nested loop or using *flatten()* method

In [29]:

```
for row in a:  
    for cell in row:  
        print(cell)
```

0
1
2
3
4
5
6
7
8
9
10
11

For loop with flatten

In [30]:

```
for cell in a.flatten():  
    print(cell)
```

0
1
2
3
4
5
6
7
8
9
10
11



nditer() method

- The function `nditer()` is a helping function that can be used from very basic to very advanced iterations

C style ordering

```
In [31]: for x in np.nditer(a, order='C'):  
         print(x)
```

0
1
2
3
4
5
6
7
8
9
10
11

Fortran style ordering

```
In [32]: for x in np.nditer(a, order='F'):  
         print(x)
```

0
4
8
1
5
9
2
6
10
3
7
11



Matplotlib

- Most of the Matplotlib utilities lies under the pyplot submodule, and are usually imported under the plt alias

```
import matplotlib.pyplot as plt
%matplotlib inline
```



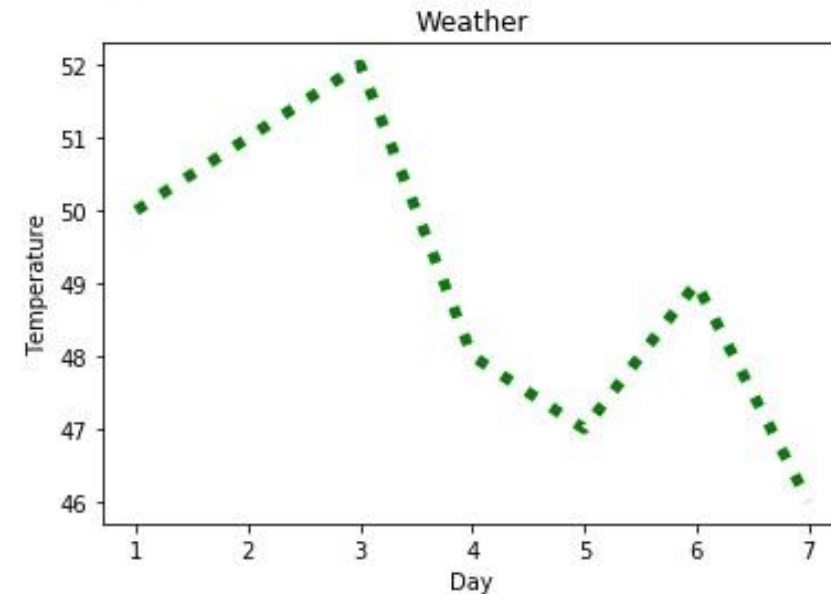
Plotting Array Values

- Plotting temperature vs day
- Putting label and title
- We can try different linestyle

```
In [13]: x=[1,2,3,4,5,6,7]  
         y=[50,51,52,48,47,49,46]
```

```
In [21]: plt.xlabel('Day')  
         plt.ylabel('Temperature')  
         plt.title('Weather')  
         plt.plot(x,y,color='green',linewidth=5, linestyle='dotted')
```

```
Out[21]: [<matplotlib.lines.Line2D at 0x16ff68db8d0>]
```

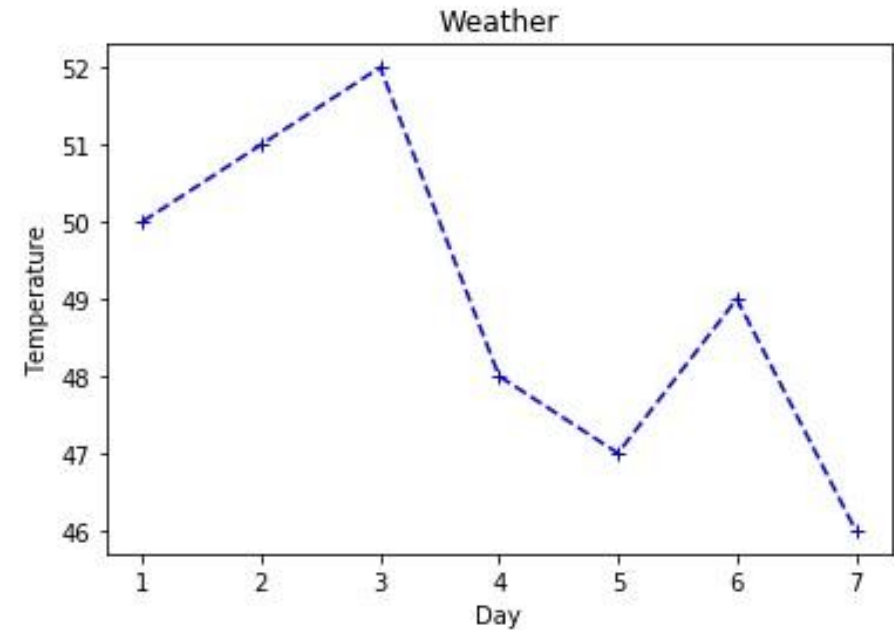


Plotting Array Values

- using format string b+
- It represent colour=blue and marker=+

```
In [65]: plt.xlabel('Day')  
plt.ylabel('Temperature')  
plt.title('Weather')  
  
plt.plot(x,y,'b+--')
```

```
Out[65]: [<matplotlib.lines.Line2D at 0x1c806e436d8>]
```



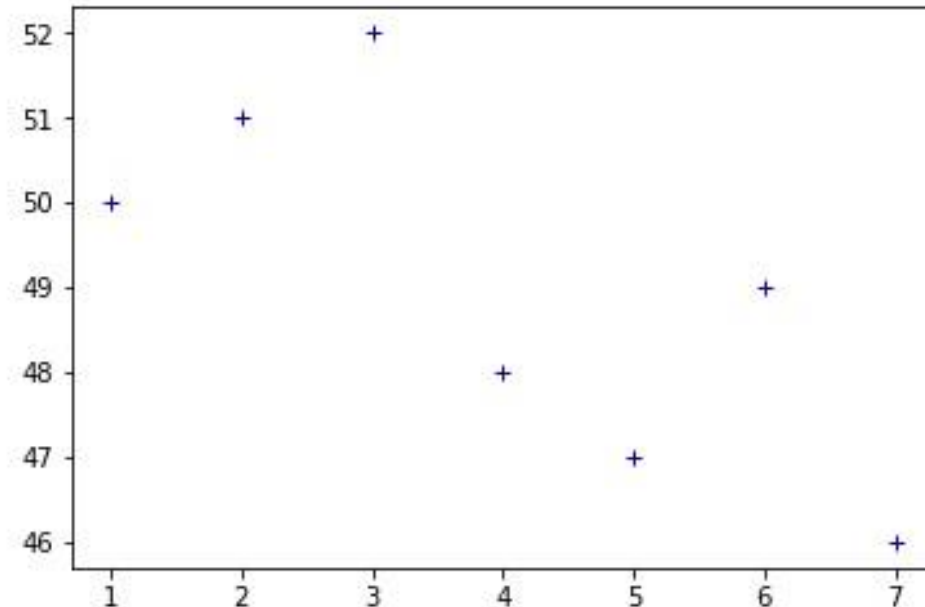
Plotting Array Values

- possible to use different marker

In [62]:

```
# Same effect as 'b+' format string  
plt.plot(x,y,color='blue',marker='+',linestyle='')
```

Out[62]: [<matplotlib.lines.Line2D at 0x1c806da3898>]



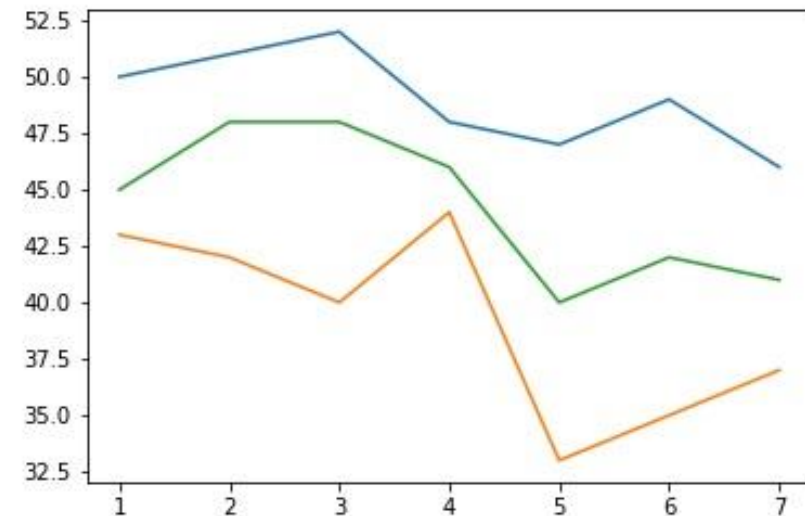
Matplotlib Lines

- possible to plot multiple lines all together

```
days=[1,2,3,4,5,6,7]  
max_t=[50,51,52,48,47,49,46]  
min_t=[43,42,40,44,33,35,37]  
avg_t=[45,48,48,46,40,42,41]
```

```
plt.plot(days, max_t, label="max")  
plt.plot(days, min_t, label="min")  
plt.plot(days, avg_t, label="average")
```

```
[<matplotlib.lines.Line2D at 0x1f851056eb8>]
```



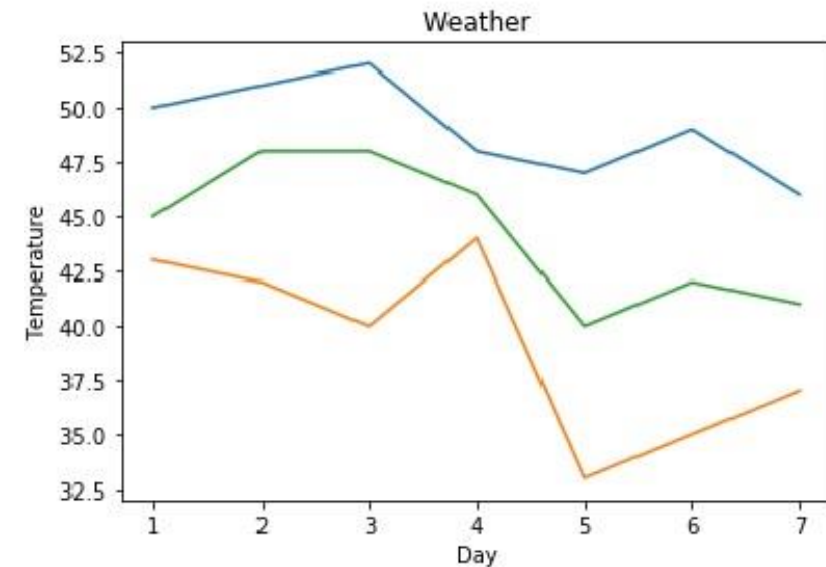
Matplotlib Lines

- putting label and title

```
# set axes labels and chart title
plt.xlabel('Day')
plt.ylabel('Temperature')
plt.title('Weather')

plt.plot(days, max_t, label="max")
plt.plot(days, min_t, label="min")
plt.plot(days, avg_t, label="average")
```

[<matplotlib.lines.Line2D at 0x1f8510c2eb8>]

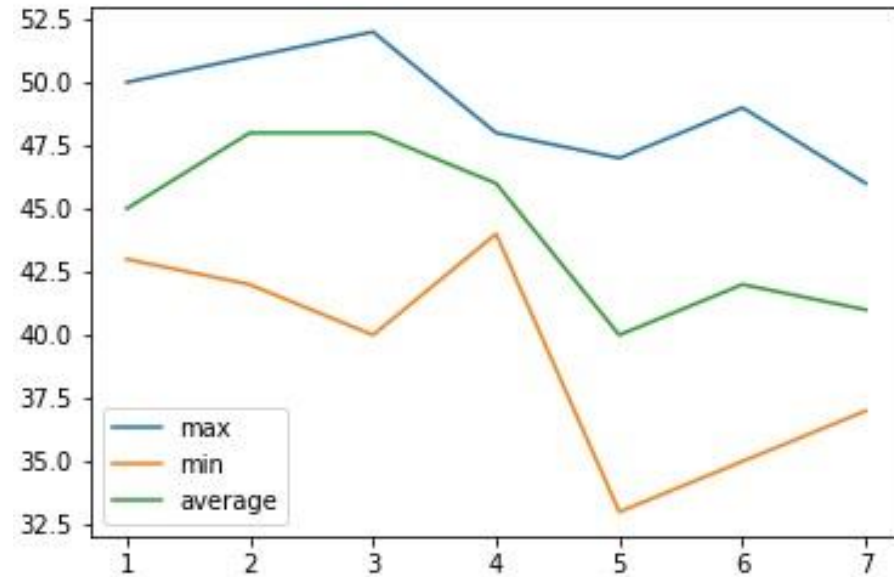


Matplotlib Lines

- putting legend
- Loc='best' will put the legend to the best possible place

```
# Show Legend  
plt.plot(days, max_t, label="max")  
plt.plot(days, min_t, label="min")  
plt.plot(days, avg_t, label="average")  
  
plt.legend(loc='best')
```

<matplotlib.legend.Legend at 0x1f85224ed30>



Matplotlib Bars

- With Pyplot, we can use the `bar()` function to draw bar graphs

```
company=['GOOGL','AMZN','MSFT','FB']  
revenue=[90,136,89,27]
```

```
xpos = np.arange(len(company))  
xpos
```

```
array([0, 1, 2, 3])
```

```
plt.bar(xpos,revenue, label="Revenue")  
  
plt.xticks(xpos,company)  
plt.ylabel("Revenue(Bln)")  
plt.title('US Technology Stocks')  
plt.legend()
```

Out[22]: <matplotlib.legend.Legend at 0x1fdce45b240>



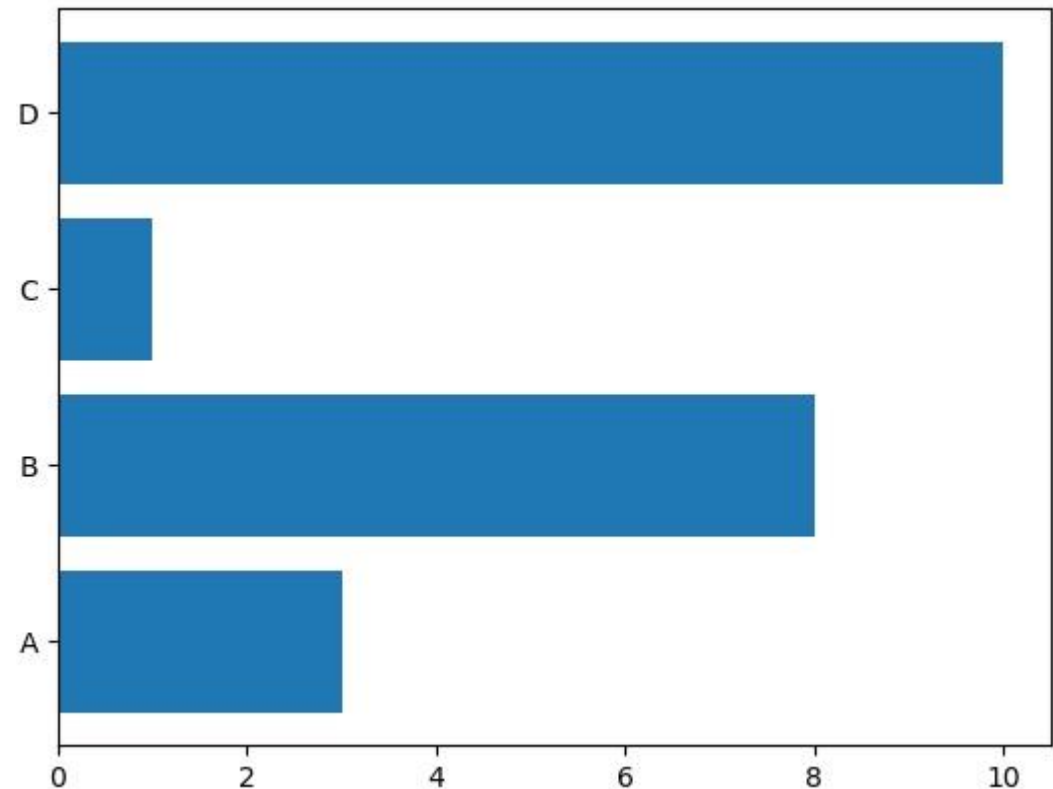
Horizontal Bars

- If you want the bars to be displayed horizontally instead of vertically, use the `barh()` function

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.barh(x, y)
plt.show()
```



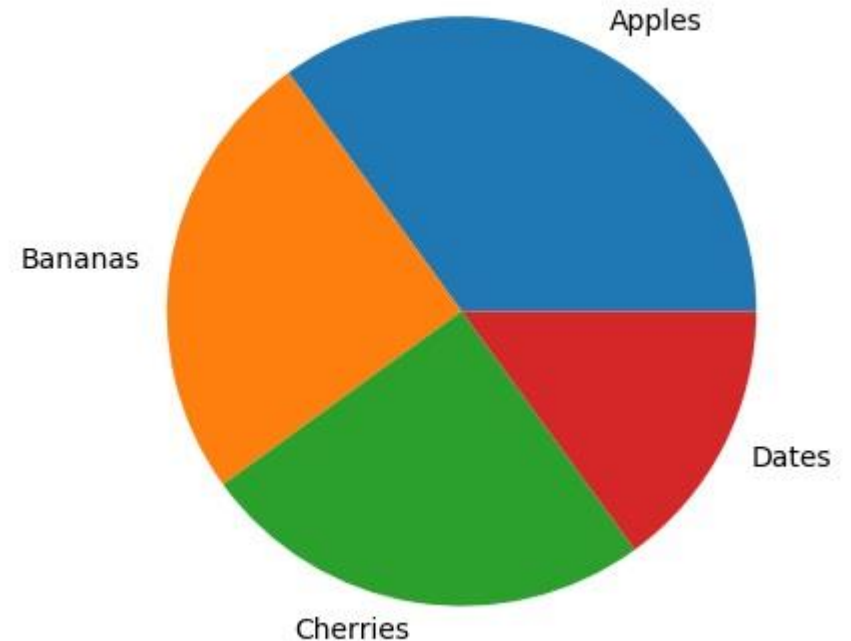
Matplotlib Pie Charts

- With Pyplot, you can use the `pie()` function to draw pie charts

```
import matplotlib.pyplot as plt
import numpy as np

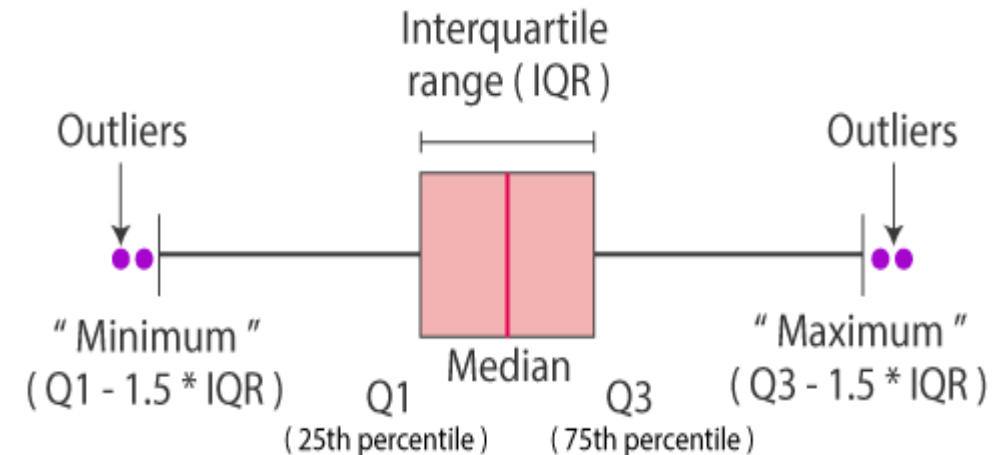
y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.show()
```



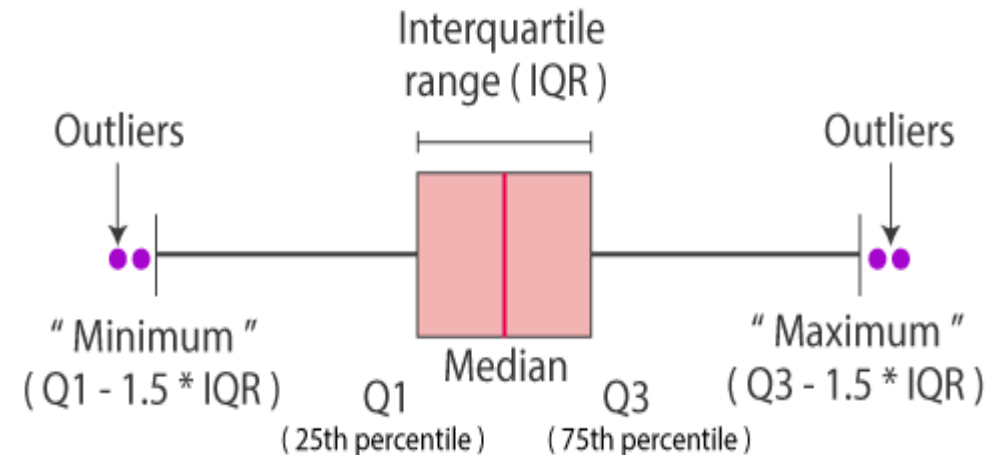
Box and Whisker Plots: Outlier Detection

- Box and Whisker Plots: Indication of how the values in the data are spread out
- Displays the five-number summary of a set of data. The five-number summary is the minimum, first/lower quartile (Q1), median (Q2/ second quartile), third/upper quartile (Q3), and maximum
- **First Quartile (Q1):** The value below which 25% of the data falls when sorted in ascending order. It's also known as the 25th percentile
- **Second Quartile (Q2):** The median of the dataset, representing the 50th percentile.
- **Third Quartile (Q3):** The value below which 75% of the data falls, also known as the 75th percentile
- The difference between Q3 and Q1 is called the Inter-Quartile Range or IQR.
- $IQR = Q3 - Q1$



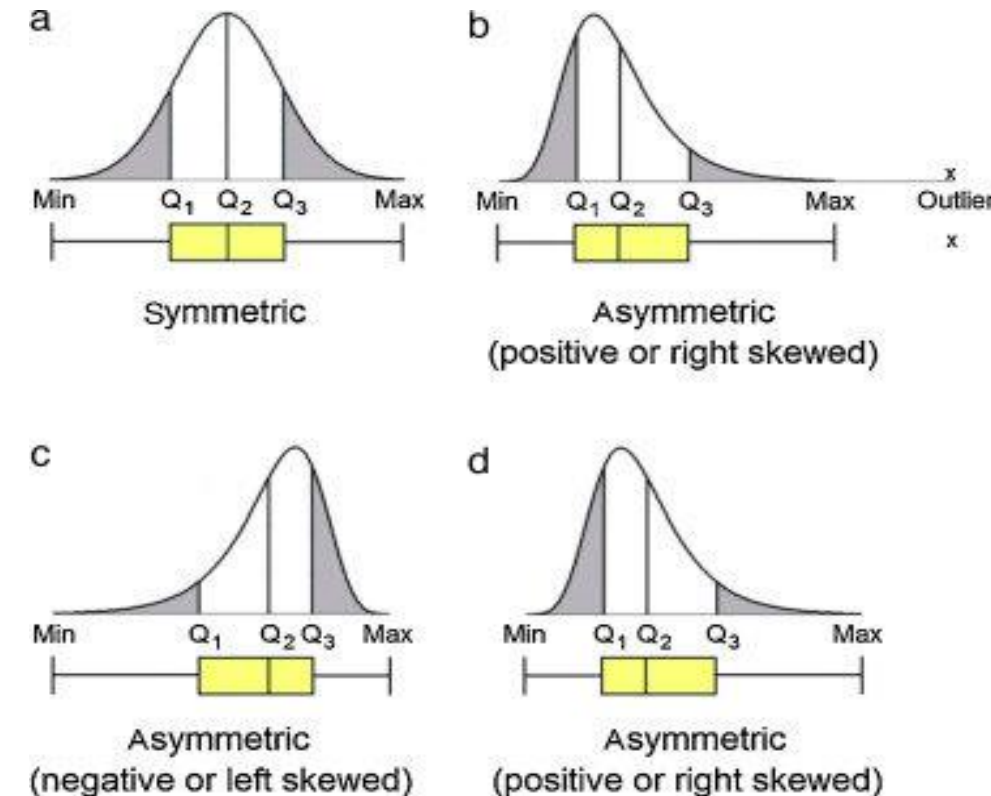
Box and Whisker Plots: Outlier Detection

- Any data point lying outside this range is considered as outlier. The range is as given below:
- Lower Bound: $(Q1 - 1.5 \times IQR)$
- Upper Bound: $(Q3 + 1.5 \times IQR)$
- Any data point less than the Lower Bound or more than the Upper Bound is considered as an outlier.



Box Plot: Skewness

- **Symmetric distribution:** left side of the distribution mirrors the right side.
- A symmetric distribution is never skewed; skewness is zero
- Symmetric distribution: $\text{mean} \cong \text{median} \cong \text{mode}$
- **Skewness refers to the tendency of a distribution to be asymmetric.**
- Skewness is lack of symmetry
- **Positive skewness** indicates a right-skewed distribution
- **Negative skewness** indicates a left-skewed distribution



Detecting and Treating Outliers

- Outliers are those data points which **differs significantly from other observations** present in given dataset
 - An outlier may occur due to the **variability** in the data, or due to **experimental/human error**
1. Deleting observations: Trimming/ Removing the Outliers
 2. Log Transforming values
 3. Mean/median Imputation
 4. Quantile based flooring and capping: In this technique, the outlier is capped at a certain value above the 90th percentile value or floored at a factor below the 10th percentile value. The data points that are less than the 10th percentile are placed with the 10th percentile value, and the data points that are greater than the 90th percentile are replaced with the 90th percentile value

